

Topo-HetGNN

Combining Meta-Path Discovery and Persistent
Homology for Heterogeneous Graph Learning

Team | j-hop

22기 박준영 장건희 23기 윤현주

KUBIG
DATA SCIENCE & AI

CONTENTS

01

Introduction

연구 배경 및 필요성
배경 지식
연구 목적 및 핵심 질문

02

Methodology

전체 파이프라인
실험 매트릭스
데이터셋
평가 지표 & 재현성 설정

03

Results

메인 성능 테이블
noisy baseline 비교
HAN vs. RGCN 비교
데이터셋별 패턴 분석

04

Conclusion & Future Work

결과 요약
연구의 의의 및 시사점
한계 및 향후 연구 과제

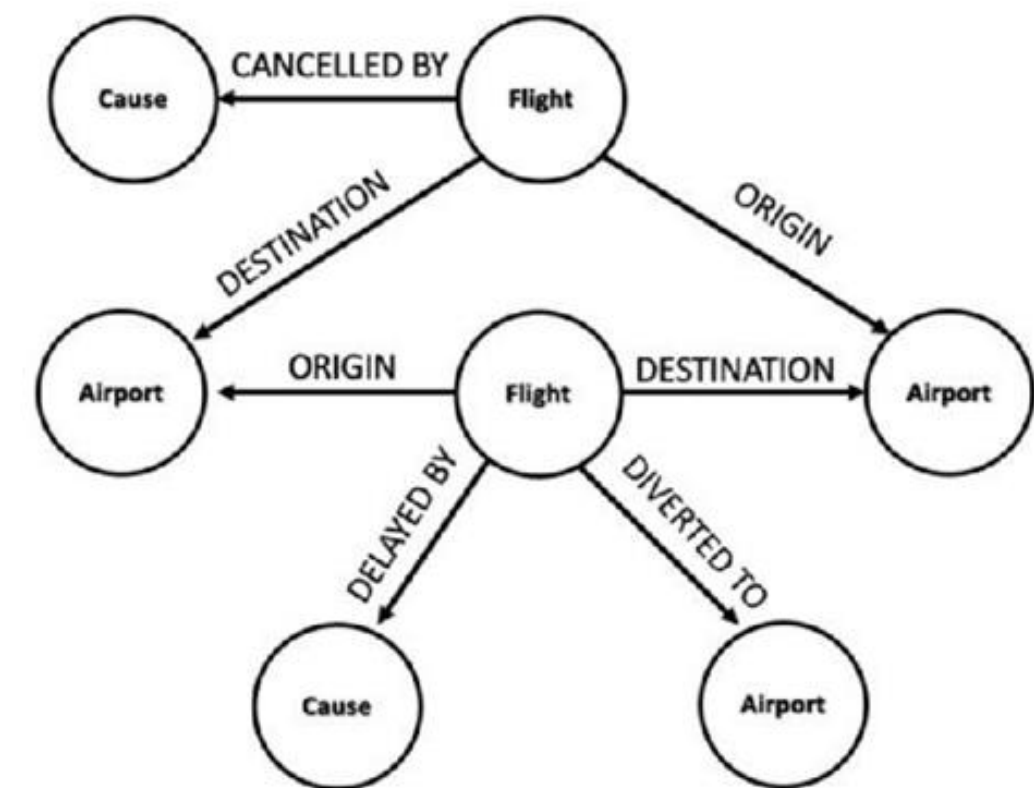
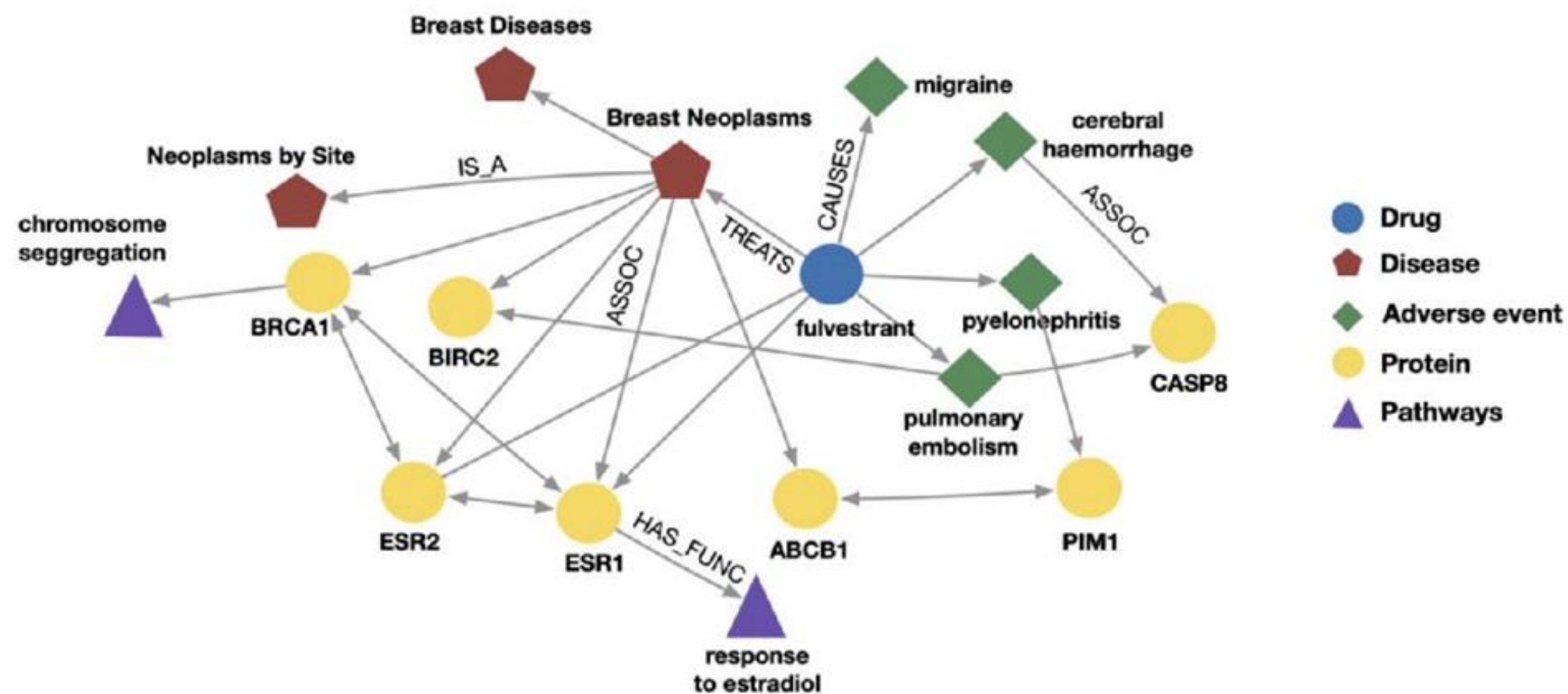


01. Introduction

01. Heterogeneous Graph

Heterogeneous Graphs란?

- 여러 종류의 노드와 엣지로 구성된 그래프
- 실세계의 많은 데이터는 Heterogeneous Graph(이종 그래프)로 표현($\leftrightarrow$ Homogeneous Graph)



01. 기존 방법론과 한계

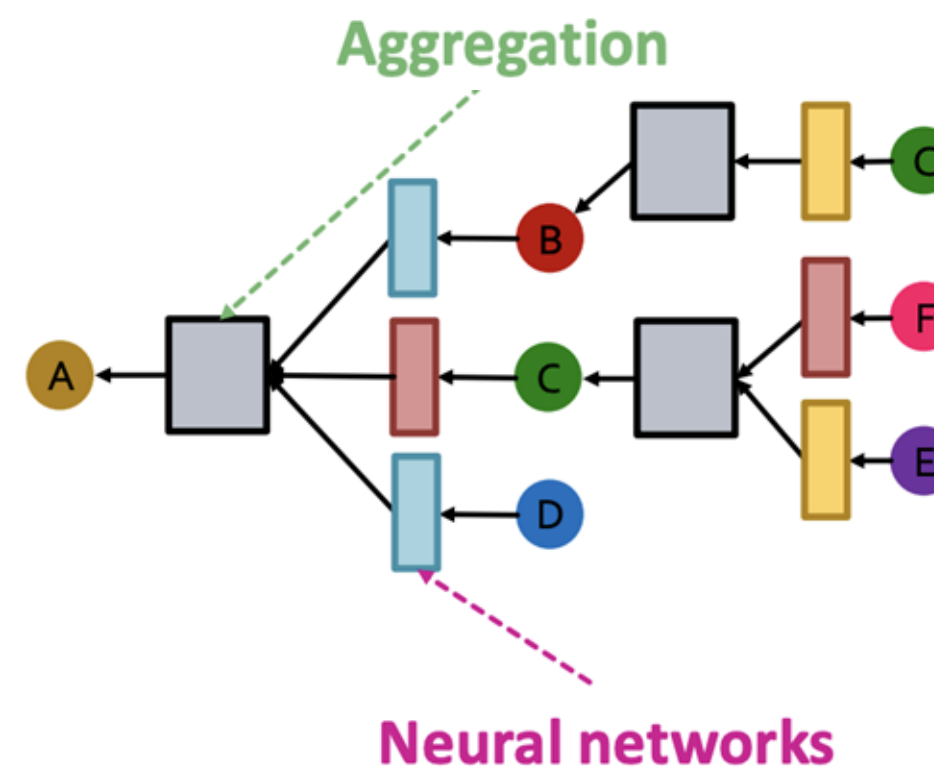
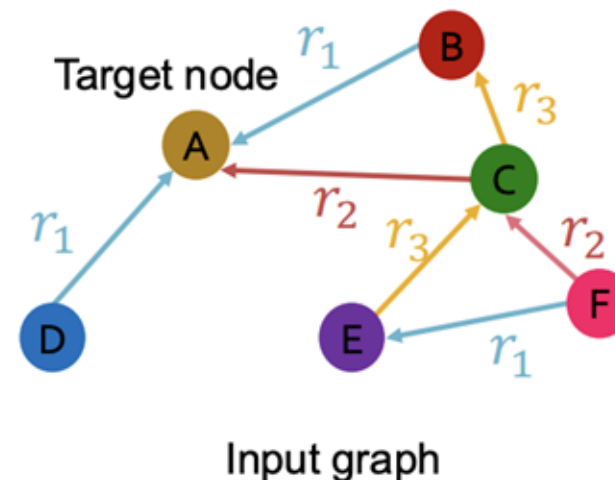
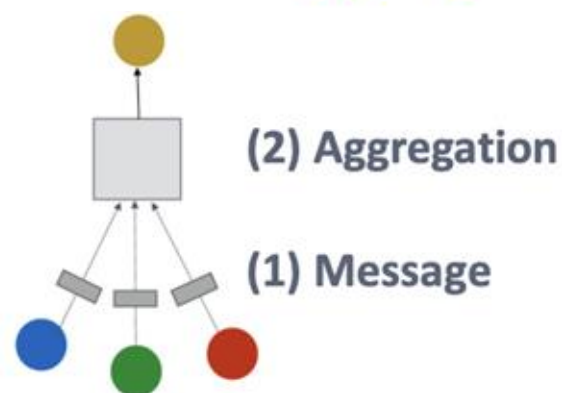
기존 GNN은 local neighborhood aggregation에 머물러 global 위상 구조를 반영하지 않는다.

기존 Het GNN의 작동 방식: Message Passing

- 각 노드는 이웃 노드의 feature를 aggregate(sum, min, max 등)하여 자신의 feature update
- 대표 방법: HAN, RGCN
- Aggregation 반경 = Layer 수 × hop (2~3 hop)

$$\mathbf{h}_v^{(l)} = \sigma \left(\underbrace{\sum_{u \in N(v)} \mathbf{w}^{(l)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}}_{\text{Aggregation}} \right)$$

Message



01. 기존 방법론과 한계

기존 GNN은 local neighborhood aggregation에 머물러 global 위상 구조를 반영하지 않는다.

기존 Het GNN의 작동 방식: Message Passing

- 각 노드는 이웃 노드의 feature를 aggregate하여 자신의 feature update
- 대표
- Aggre

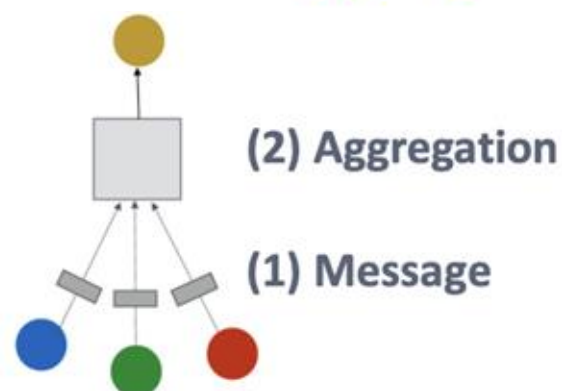
한계점

1. **Locality**: aggregation 반경 밖의 구조 정보 무시
2. **Topology Blindness**: 그래프의 루프, 구멍, Cluster 연결성 등 Global Topology구조를 feature에 담지 못함

3. **Heterogeneity**: Topology 분석 도구 대부분이 Homogeneous Graph만 지원

$$\mathbf{h}_v^{(l)} =$$

$\{u \in N(v)\}$
Aggregation

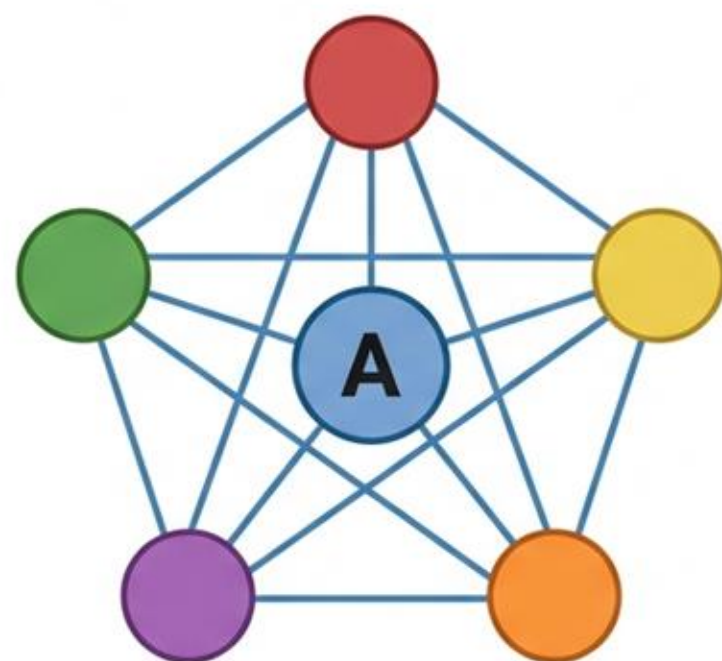


01. 기존 방법론과 한계

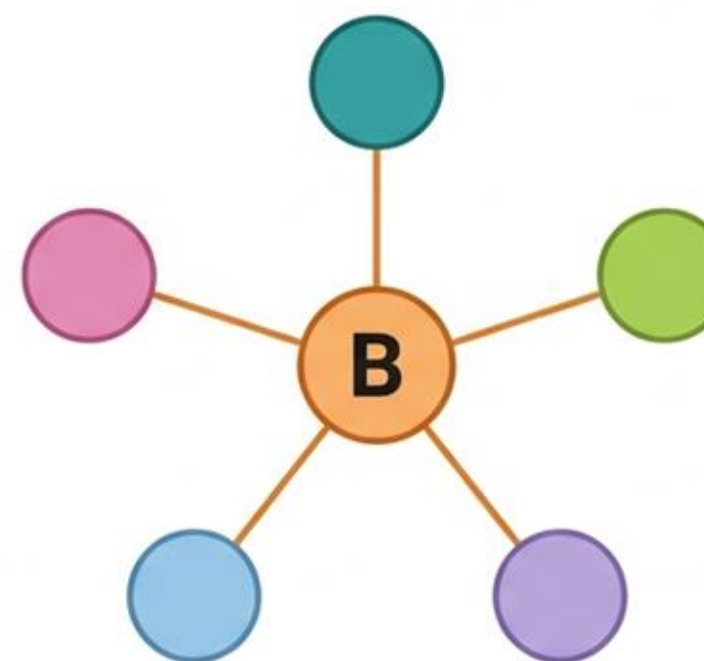
GNN message passing은 local neighborhood 정보만 aggregate한다.

그러나 같은 이웃 구조를 가진 두 노드라도, 그래프 전체에서의 전역 위상 구조는 다를 수 있다.

Author A – Clustered Co-authors



Author B – Dispersed Co-authors

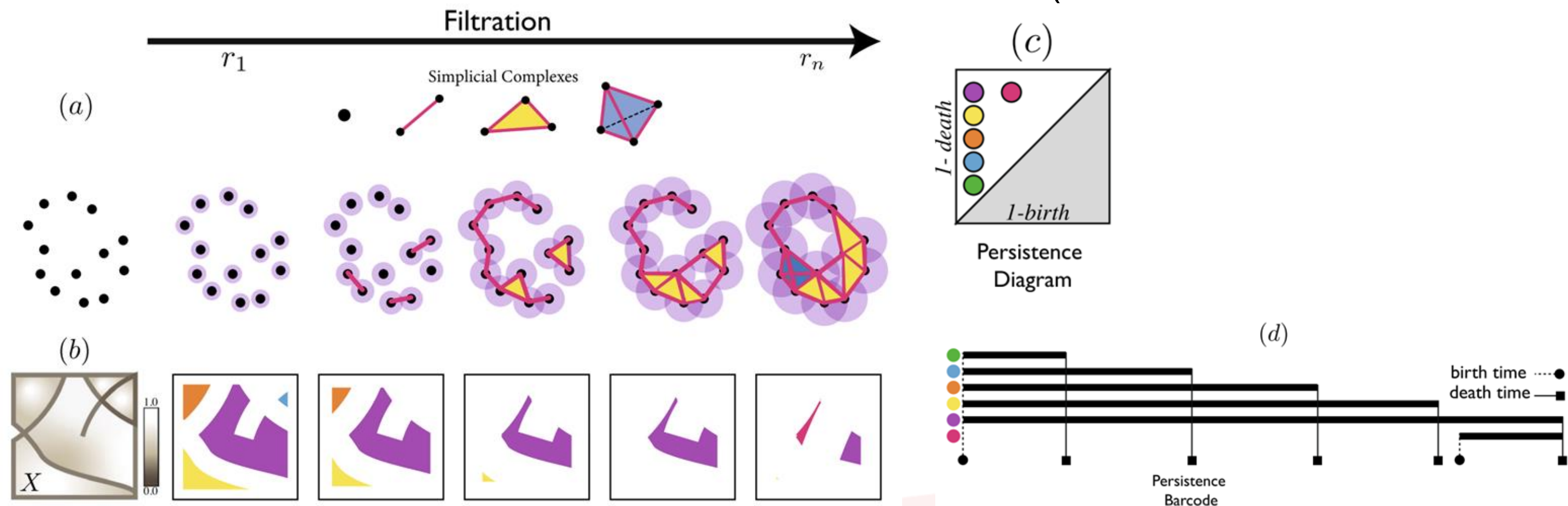


“Local neighborhood aggregation (message passing) cannot distinguish these two structures, despite their different global connectivity.”

01. Persistence Homology

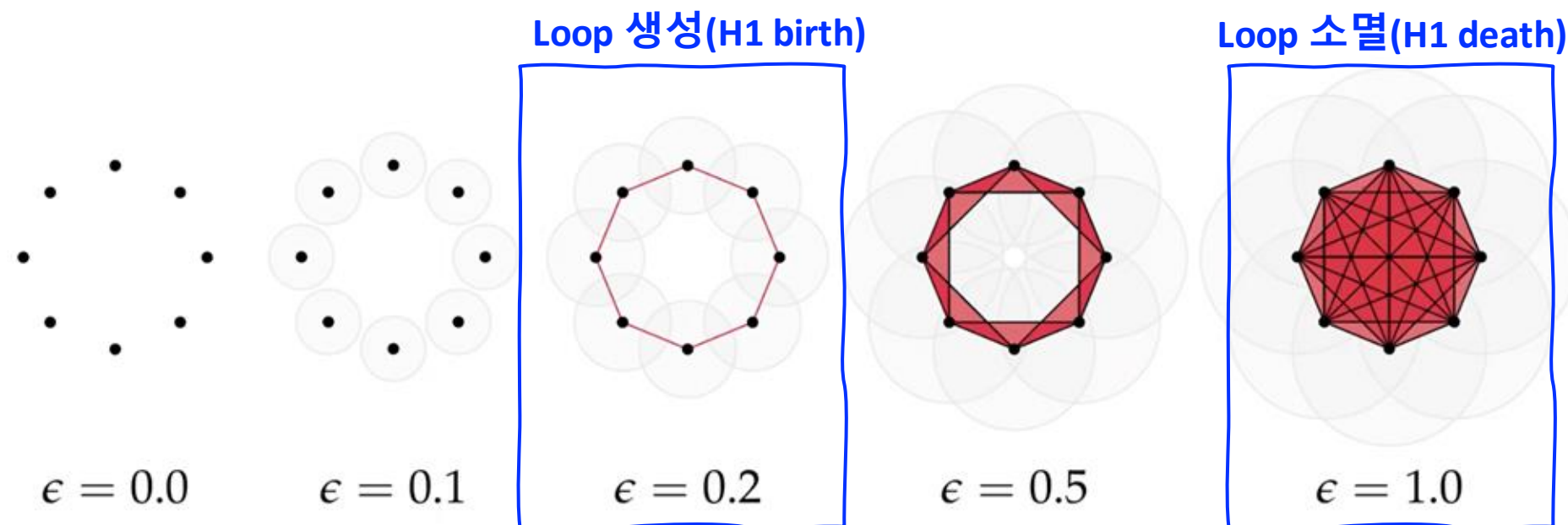
Persistence Homology: 연결 성분(H_0), 루프(H_1) 등 MPNN으로는 포착하지 못하는 그래프의 global topology를 표현

- Connected components(H_0): 그래프가 몇 개의 덩어리로 나뉘어 있는지 (노드가 큰 허브 클러스터의 일부인지/고립된 작은 그룹인지)
- Loops(H_1): 그래프 안에 몇 개의 고리(loop)구조가 있는지 (상호 연결된 서브구조인지/Tree같은

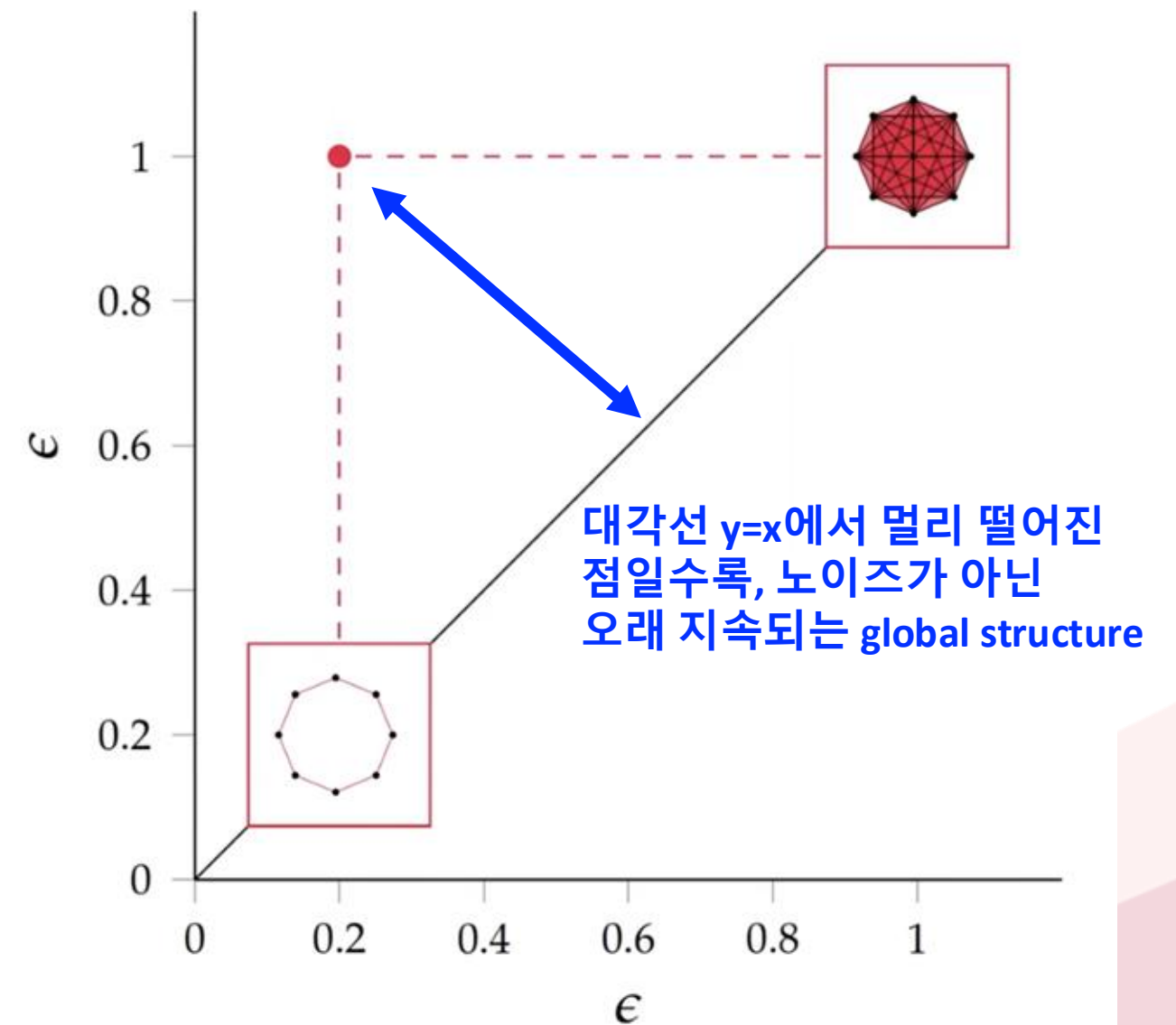


01. Persistence Homology

Persistence Homology: 연결 성분(H_0), 루프(H_1) 등 topological feature가 생긴 시점(birth)과 소멸된 시점(death)를 가지고 이를 좌표로 persistence diagram에 점으로 찍는다.



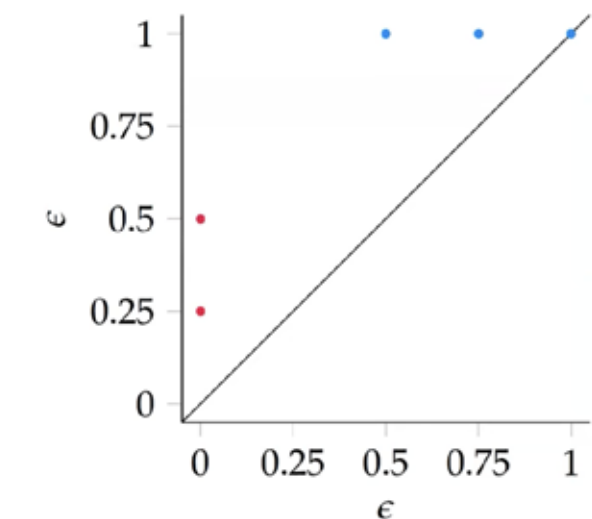
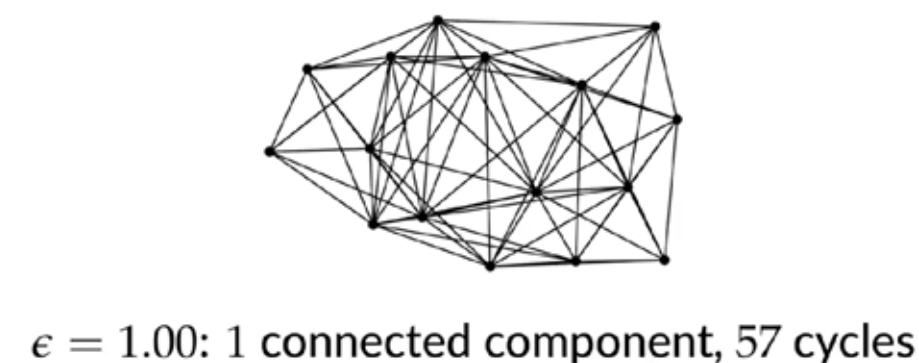
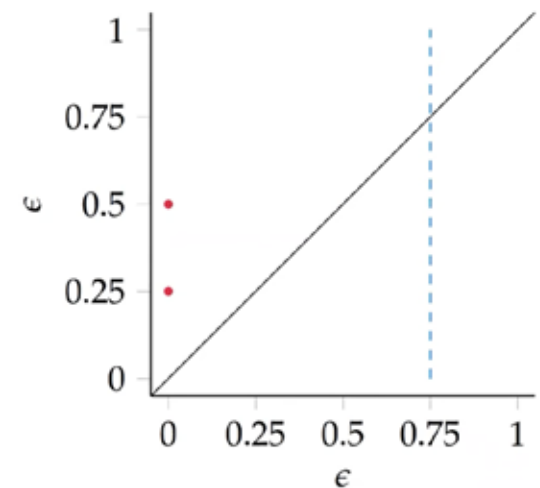
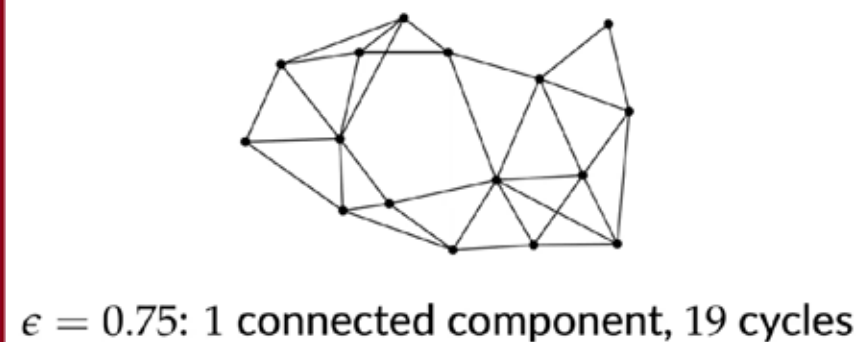
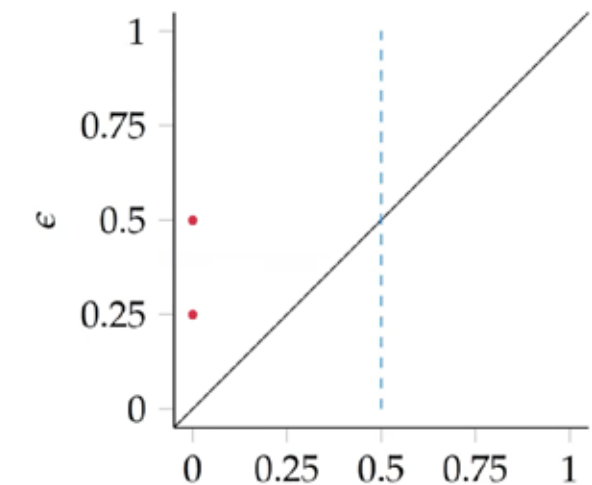
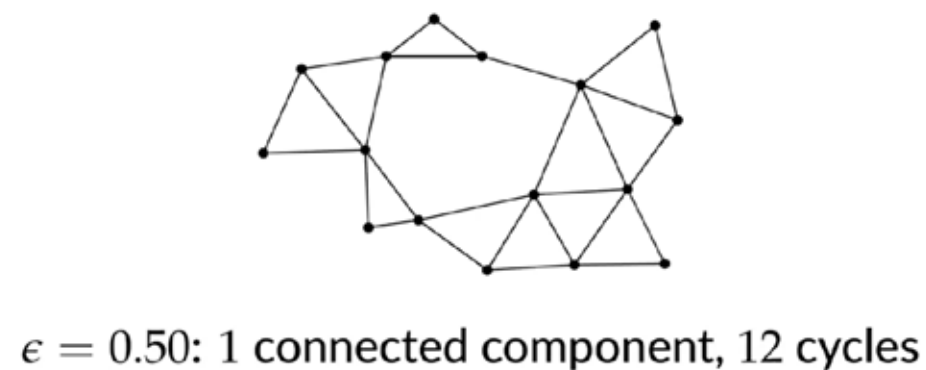
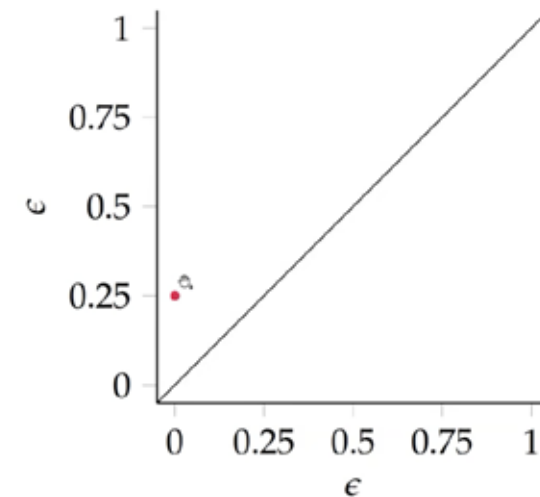
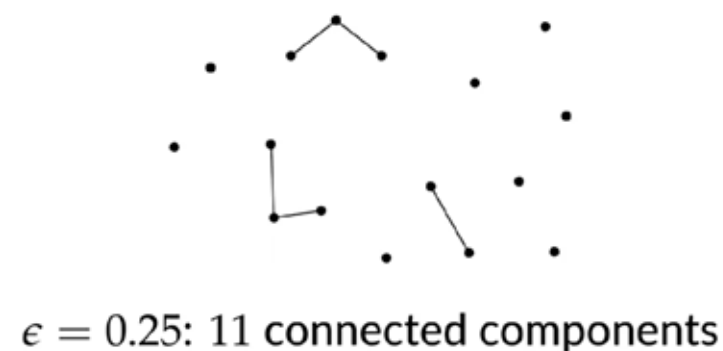
Here, the topological feature is the circle that underlies that data. Since it persists from $\epsilon = 0.20$ to $\epsilon = 1.0$, its persistence is $\text{pers} = 1.0 - 0.20 = 0.80$.



01. Persistence Homology

그래프의 노드에 실수값을 매핑하는 함수
e.g. Degree/Heat Kernel Signature

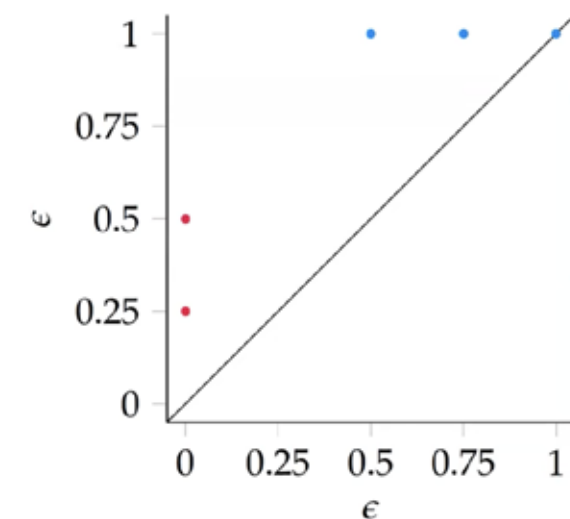
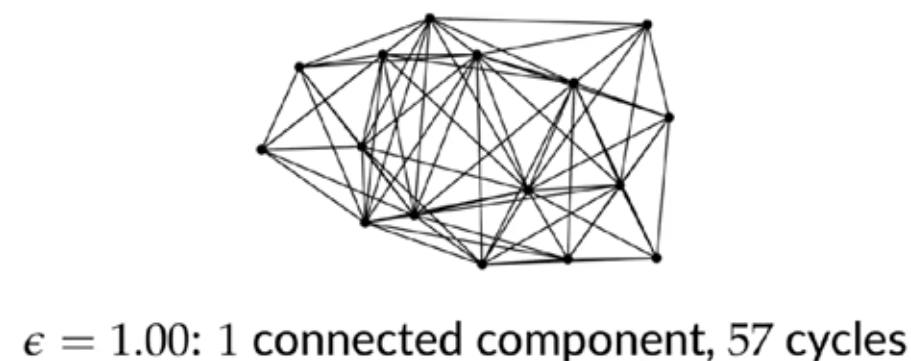
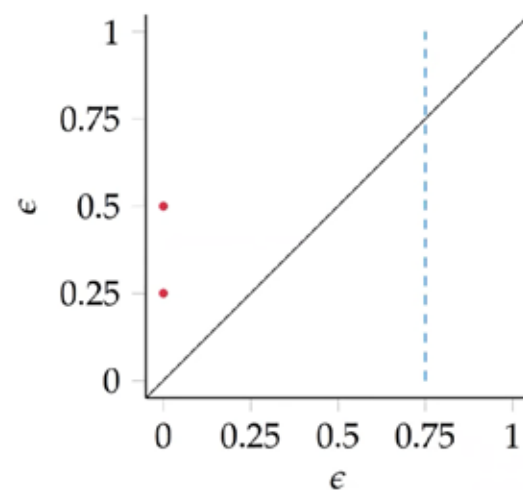
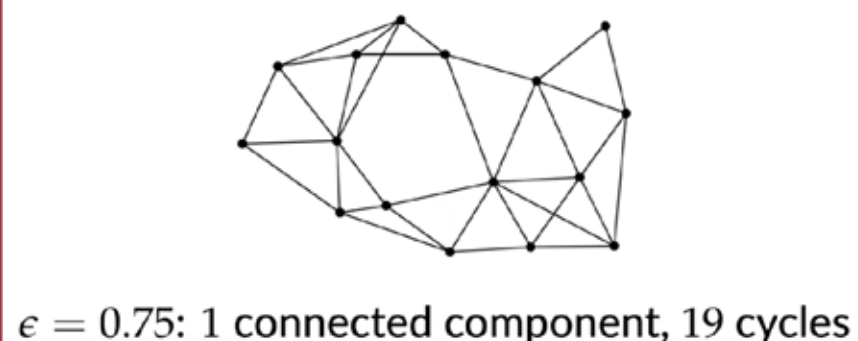
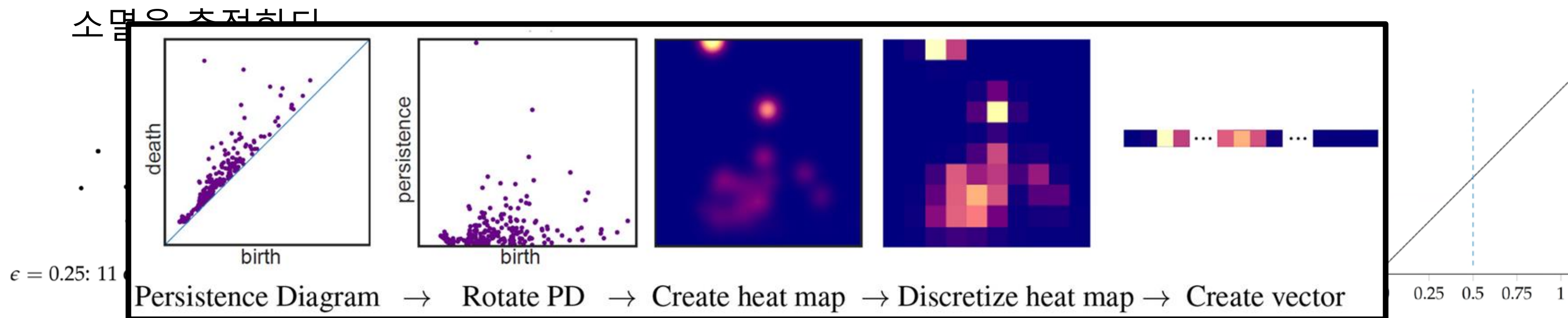
1. 그래프에 filter function f 를 각 노드 v 에 적용한 값 $f(v)$ 들을 오름차순으로 정렬
2. 정렬 순서대로 노드/엣지를 그래프에 추가해 나가면서 단계별 subgraph가 만들어진다.
3. Filtration = subgraph들의 증가하는 수열. 이렇게 노드/엣지를 누적시키면서 위상 특징의 생성과 소멸을 추적한다.



01. Persistence Homology

그래프의 노드에 실수값을 매핑하는 함수
e.g. Degree/Heat Kernel Signature

1. 그래프에 filter function f 를 각 노드 v 에 적용한 값 $f(v)$ 들을 오름차순으로 정렬
2. 정렬 순서대로 노드/엣지를 그래프에 추가해 나가면서 단계별 subgraph가 만들어진다.
3. Filtration = subgraph들의 증가하는 수열. 이렇게 노드/엣지를 누적시키면서 위상 특징의 생성과 소멸을 추적한다.



01. PDGNN (Yan et al., NeurIPS 2022)

- **역할**: EPD를 GNN의 feature로 넣기 위한 모델. 기존의 복잡한 EPD계산 과정($O(n^3)$)을 신경망으로 근사하여 빠르게 처리
- **입력**: Homogeneous Graph + HKS(Heat Kernel Signature) filter function
- **HKS**: 각 노드의 열 확산 패턴을 측정, 그래프 구조에 민감
- **출력**: 노드별 75차원 persistence image feature
- **핵심 한계**: 동종 그래프(homogeneous graph)에서만 동작
- 대부분의 그래프는 이종 그래프 → 이종 그래프는 여러 관계 타입이 혼재 → PDGNN 사용 불가

그렇다면 Heterogeneous Graph을 Homogeneous하게 만들 순 없을까?

MetaPath을 통해 Homogeneous Subgraph을 추출하자!

01. MetaPath란?

MetaPath = 이종 그래프에서 동일 타입 노드를 연결하는 관계 시퀀스

MetaPath를 따라 인접 행렬을 구성하면 **Het graph**에서 **Hom subgraph**를 만들 수 있다.

조건: MetaPath의 시작
노드와 끝 노드의 종류가
같아야한다.

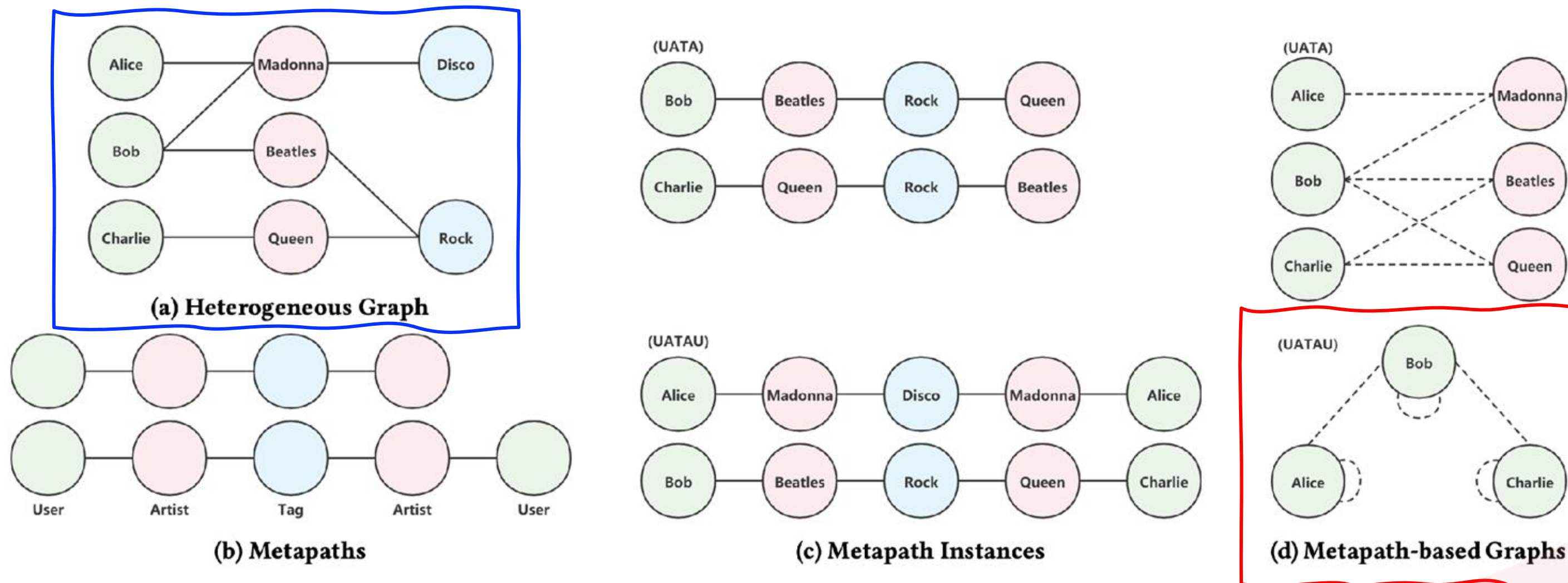


Figure 1: An illustration of the terms defined in Section 2. (a) An example heterogeneous graph with three types of nodes (i.e., users, artists, and tags). (b) The User-Artist-Tag-Artist (UATA) metapath and the User-Artist-Tag-Artist-User (UATAU) metapath. (c) Example metapath instances of the UATA and UATAU metapaths, respectively. (d) The metapath-based graphs for the UATA and UATAU metapaths, respectively.

01. GTN(Yun et al., NeurIPS 2019)

역할: MetaPath를 사람이 직접 정의하지 않고 자동으로 학습하는 네트워크

동작 원리: 여러 관계 타입의 인접 행렬에 소프트 가중치를 학습하여 조합

□ 채널별 MetaPath graph H 생성

출력: num_channels=4 개의 동종 그래프 H0, H1, H2, H3

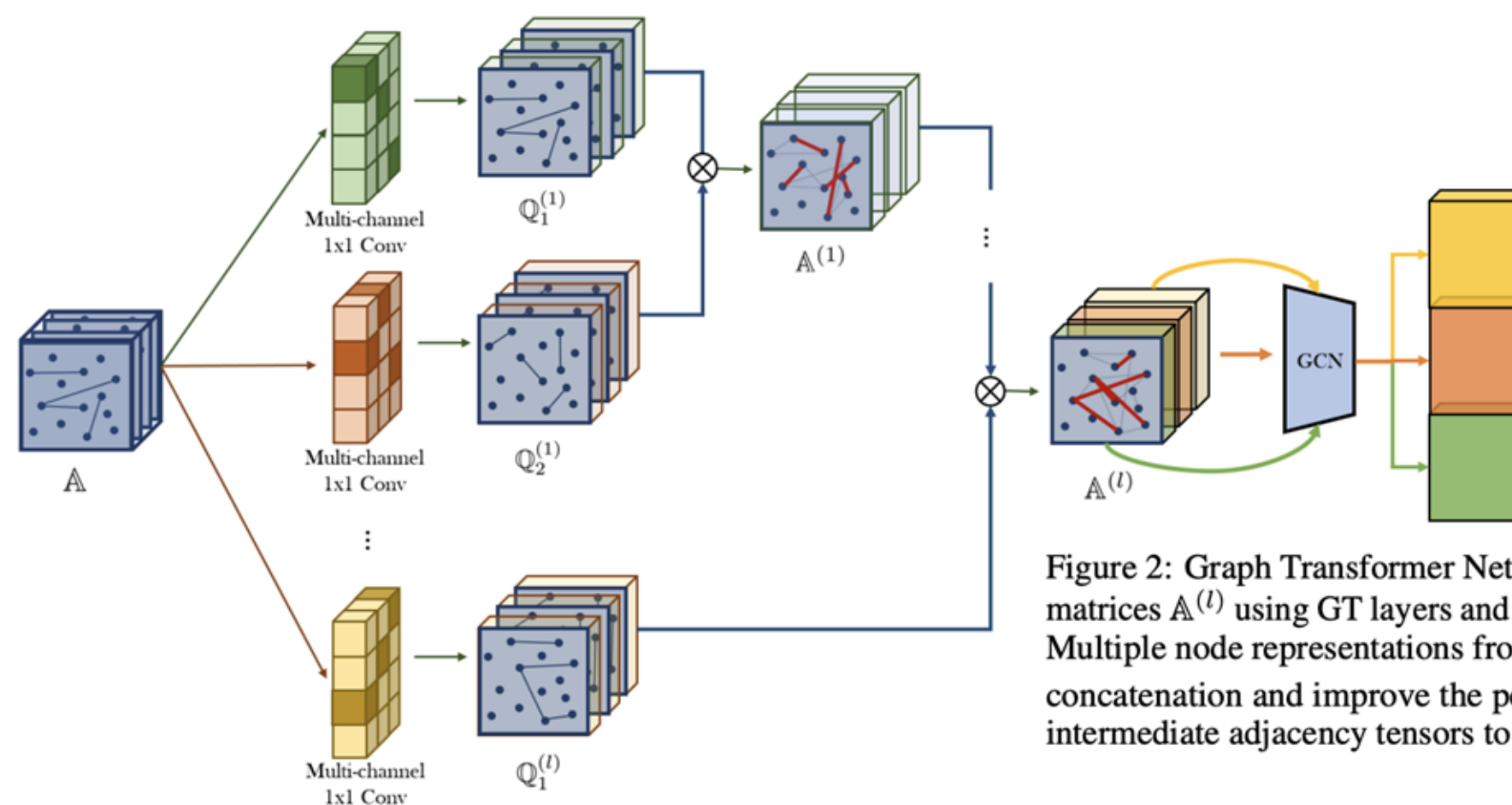
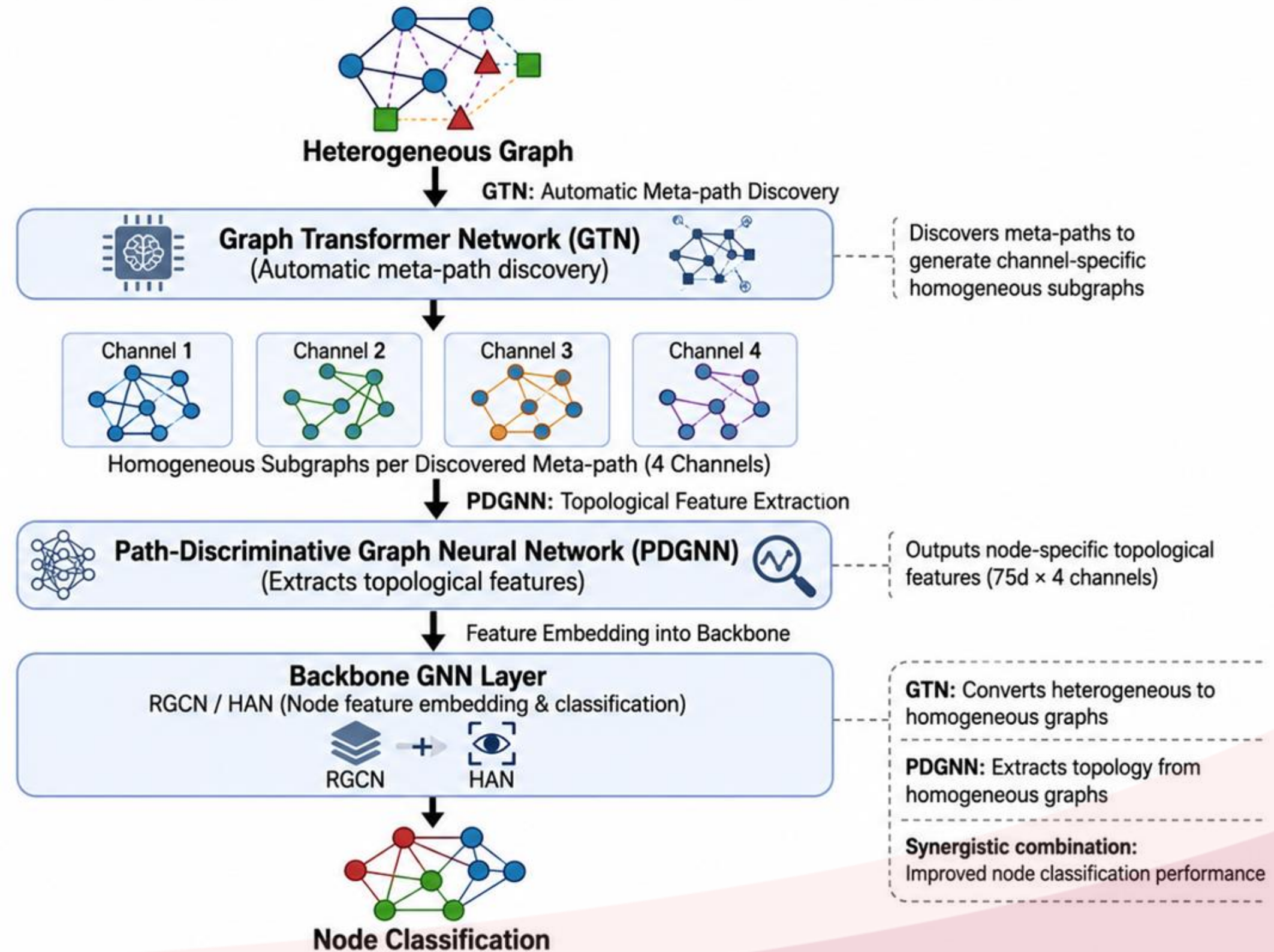


Figure 2: Graph Transformer Networks (GTNs) learn to generate a set of new meta-path adjacency matrices $\mathbb{A}^{(l)}$ using GT layers and perform graph convolution as in GCNs on the new graph structures. Multiple node representations from the same GCNs on multiple meta-path graphs are integrated by concatenation and improve the performance of node classification. $Q_1^{(l)}$ and $Q_2^{(l)} \in \mathbf{R}^{N \times N \times C}$ are intermediate adjacency tensors to compute meta-paths at the l th layer.

Dataset	Predefined Meta-path	Meta-path learnt by GTNs	
		Top 3 (between target nodes)	Top 3 (all)
DBLP	APCPA, APA	APCPA, APAPA, APA	CPCPA, APCPA, CP
ACM	PAP, PSP	PAP, PSP	APAP, APA, SPAP
IMDB	MAM, MDM	MDM, MAM, MDMDM	DM, AM, MDM

01. 연구 핵심 아이디어

Proposed Heterogeneous Graph Node Classification Pipeline



01. 연구 목적 및 핵심 질문

Q1

GTN-PDGNN을 통해 추출한 homology feature을 Heterogeneous GNN에 추가하면
Node Classification 성능이 향상되는가?

Q2

성능 향상이 있다면, 그것이 진짜 위상 신호 때문인가? 아니면 **feature dimension 증가** 효과인가?

Q3

위상 feature의 기여는 GNN backbone 구조(**HAN vs. RGCN**)에 따라 달라지는가?



02. Methodology

02. 전체 파이프라인

3단계 Staged Training

- Stage 1 → Stage 2 → Stage 3 순차 실행
- 이유: GTN·PDGNN 결합 시 gradient explosion 위험; 각 모듈 독립 최적화가 더 안정적

Stage	모듈	입력	출력
1	GTN	이종 그래프 전체	H (4, N, N) 채널별 메타패스 그래프
2	PDGNN	$H_0 \sim H_3$	노드별 위상 feature (N, $75 \times 4 \rightarrow 75$)
3	RGCN / HAN	원본 feature $\oplus$ 위상 feature	노드 분류

02. 실험 매트릭스

	wo topology	noisy topology	GTN-PDGNN topology
HAN	(a)	(b)	(c)
RGCN	(d)	(e)	(f)

(a)(d) Baseline: Heterogeneous GNN backbone만 사용

(b)(e) Noisy baseline: 같은 dimension의 random/mixed feature 주입 → feature 차원 효과만 측정

- b1/e1: Random Gaussian noise

- b2/e2: Class-wise mixing (같은 클래스 내 위상 feature 섞기 — local topology 정보 제거)

(c)(f): 실제 위상 feature 주입 □ 핵심 주장

- 실제 topology > random noise: 단순 차원 증가 이상의 효과가 있음

- 실제 topology > class-wise mixing: class 평균적 분포가 아니라, node-specific topology가 중요함

- class-wise mixing ≈ 실제 topology: topology feature가 개별 노드 구조보다는 class-level prior로만

자도해은 가느서

Noisy baseline이 왜 필요한가?

→ (f)>(e), (c)>(b)이어야만 topology feature 효과 주장 가능.
(f)≈(e), (c)≈(b) 이면 차원 증가 효과에 불과.

02. 실험 매트릭스

Backbone GNN 선택 이유: RGCN vs. HAN

Heterogeneous graph을 처리하는 GNN 모델은 두 부류 존재:

- MetaPath을 활용하는 HAN류
- MetaPath을 활용하지 않고 node type relation을 활용하는 더 기본적인 RGCN류

	HAN	RGCN
메타패스 처리	semantic attention (강)	relation별 W_r 집계 (약)
위상 기여 예상	이미 메타패스 Attention → 기여 제한	메타패스 주의 없음 → 위상 기여 더 명확

- 기존의 GNN모델 종류별 위상 feature의 기여를 비교해볼 수 있음.



03. Results

03. 결과

데이터셋	(a) HAN	(b1) +noise	(b2) +mix	(c) +위상
acm	0.895±0.006	0.887±0.008	0.899±0.006	0.894±0.008
dblp	0.937±0.005	0.938±0.004	0.956±0.008	0.938±0.003
imdb	0.438±0.009	0.441±0.004	0.402±0.013	0.402±0.015
mag	0.017±0.012	0.026±0.011	0.019±0.009	0.023±0.007
aifb	0.451±0.040	0.478±0.100	0.538±0.106	0.575±0.145

데이터셋	(d) RGCN	(e1) +noise	(e2) +mix	(f) +위상
acm	0.925±0.006	0.919±0.013	0.923±0.012	0.903±0.006
dblp	0.934±0.004	0.930±0.008	0.939±0.005	0.859±0.017
imdb	0.636±0.004	0.579±0.008	0.575±0.033	0.566±0.035
mag	0.104±0.053	0.090±0.034	0.088±0.048	0.066±0.036
aifb	0.752±0.018	0.673±0.071	0.720±0.134	0.643±0.287

- **Concat** 주입 시 backbone(HAN, RGCN)에서 성능 하락
(HAN에서의 aifb 성능 향상은 homology 정보 때문이 아님)
- 심지어 random noise 보다는 성능이 떨어지는 경우 존재 → aggregate할 때 노이즈와 달리 평균으로 소거 되지 않고 topological feature “여기에 loop 많다” 와 같은 구조 신호인데 node feature space와 concat을 하게 되면 구조 상관 잡음이 발생하고
- 그렇다면 node의 정보와 구조의 정보를 mixing하는 concat이 아닌 구조 정보를 구조 제어에 사용하여 gating하는 방법이 더 적절

03. gate vs concat

$$h'_u = W_0 h_u + \sum_{r \in \mathcal{R}} \frac{1}{|N_r(u)|} \sum_{v \in N_r(u)} (W_r h_v) \odot g_{uv}$$

$$g_{uv} = \sigma(\text{MLP}([\tilde{g}_u || \tilde{g}_v])) \in (0, 1)^d$$

$$t_u^{(1)}, t_u^{(2)}, \dots, t_u^{(C)} \in \mathbb{R}^{75}$$

$$w_c = \frac{1}{N} \sum_u q^\top \tanh(W t_u^{(c)} + b)$$

$$\beta = \text{softmax}(w_1, \dots, w_C)$$

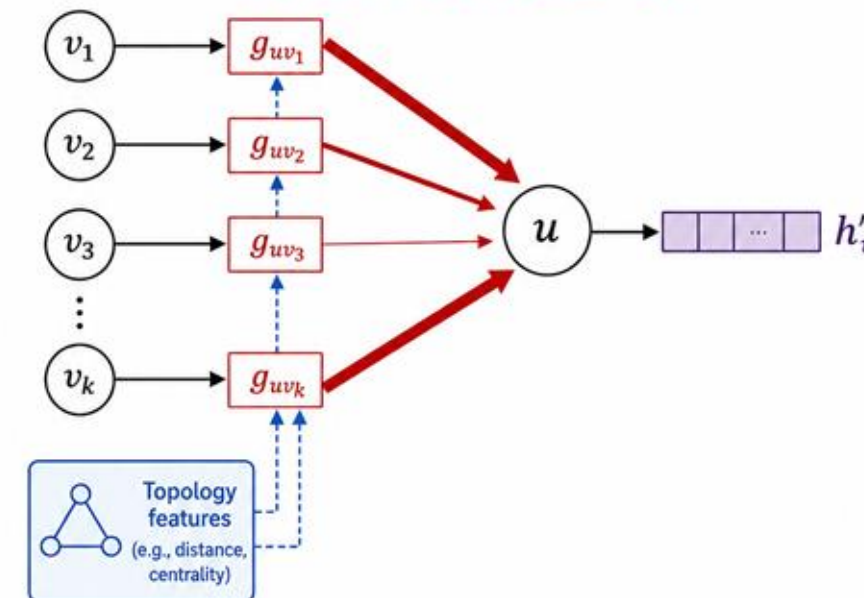
$$\tilde{g}_u = \sum_{c=1}^C \beta_c t_u^{(c)} \in \mathbb{R}^{75}$$

$$\tilde{x}_u = [x_u || \tilde{g}_u] \in \mathbb{R}^{F+75}$$

$$W_r \tilde{x}_v = \underbrace{W_r^{(x)} x_v}_{\text{원본 몫}} + \underbrace{W_r^{(g)} \tilde{g}_v}_{\text{위상 몫}}$$

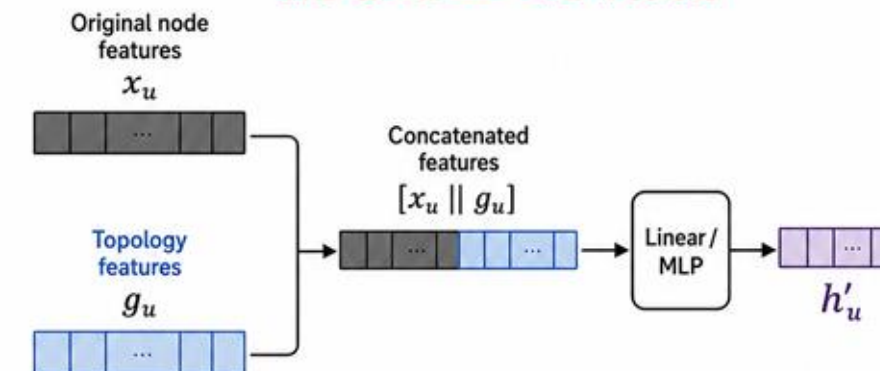
1. Gate: 위상 정보가 메시지를 조절

위상 정보가 메시지 강도를 제어



2. Concat: 위상 정보를 특징에 추가

위상 정보를 노드 특징에 덧붙임

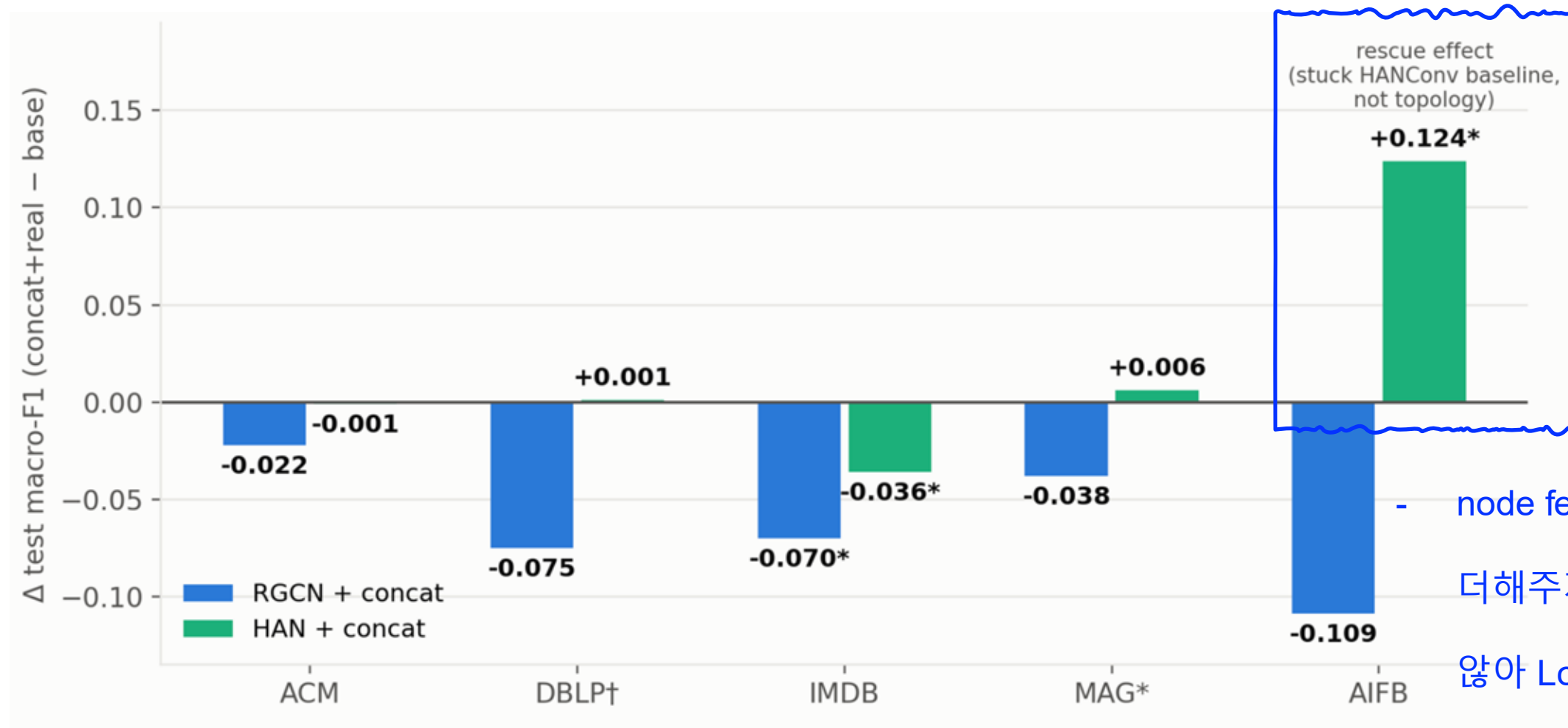


03. gate vs concat 결과

데이터셋	RGCN	concat+real	gate+noise	gate+mix	gate+real
acm	0.925±0.006	0.903±0.006	0.924±0.011	0.934±0.007	0.932±0.010
dblp	0.934±0.004	0.859±0.017	0.923±0.007	0.962±0.004	0.938±0.003
imdb	0.636±0.004	0.566±0.035	0.629±0.005	0.637±0.005	0.640±0.005
mag	0.104±0.053	0.066±0.036	0.058±0.017	0.091±0.047	0.082±0.033
aifb	0.752±0.018	0.643±0.287	0.684±0.065	0.763±0.013	0.752±0.014

데이터셋	HAN	concat+real	gate+noise	gate+mix	gate+real
acm	0.895±0.006	0.894±0.008	0.867±0.009	0.884±0.006	0.887±0.004
dblp	0.937±0.005	0.938±0.003	0.924±0.009	0.936±0.007	0.934±0.005
imdb	0.438±0.009	0.402±0.015	0.464±0.010	0.500±0.011	0.491±0.017
mag	0.017±0.012	0.023±0.007	0.032±0.015	0.036±0.015	0.037±0.017
aifb	0.451±0.040	0.575±0.145	0.585±0.087	0.724±0.139	0.722±0.144

03. gate vs concat 결과

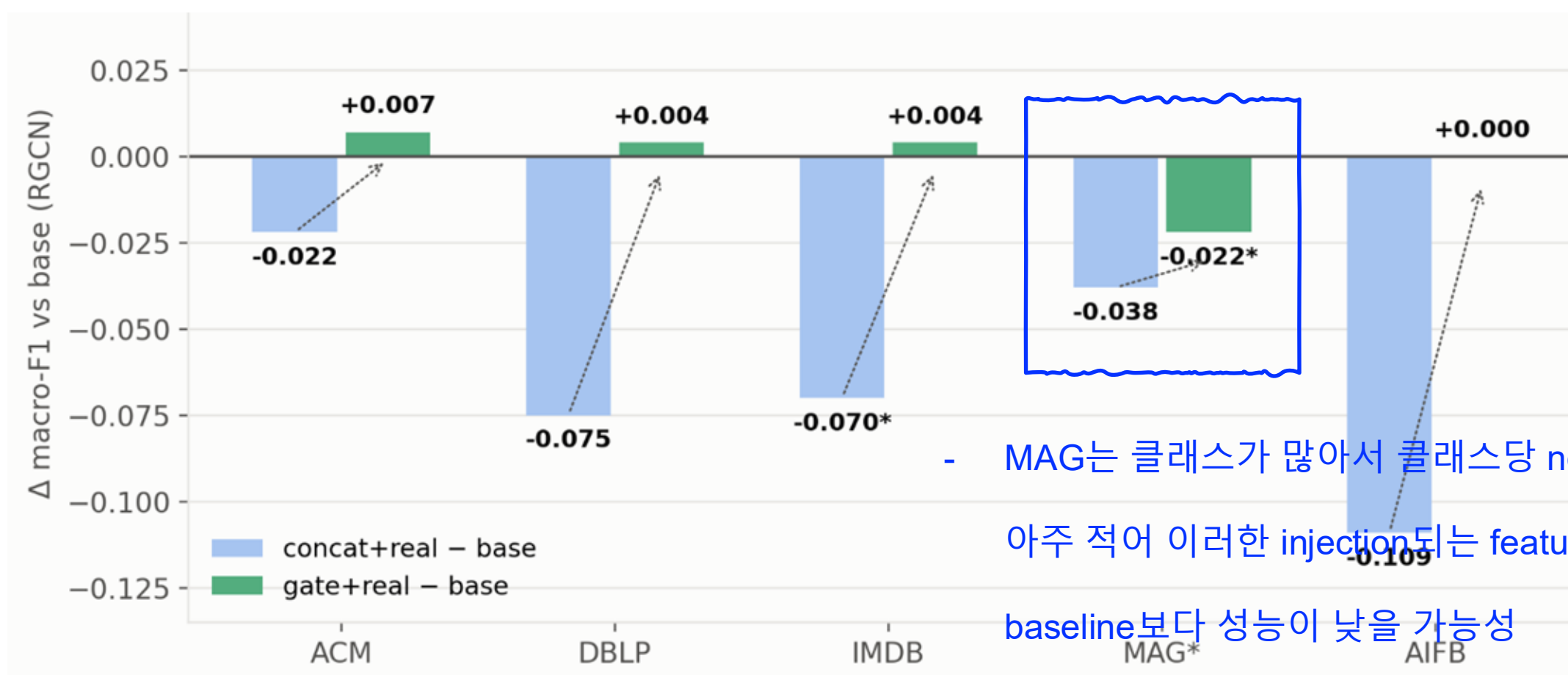


1. Concat은 유해하다

- RGCN backbone에서는 전부 하락
- RGCN은 관계별 선형변환 + 평균 aggregate여서 잡음을 거를 장치 존재하지

않아 성능 하강이 HAN에 비해 뚜렷

03. gate vs concat 결과

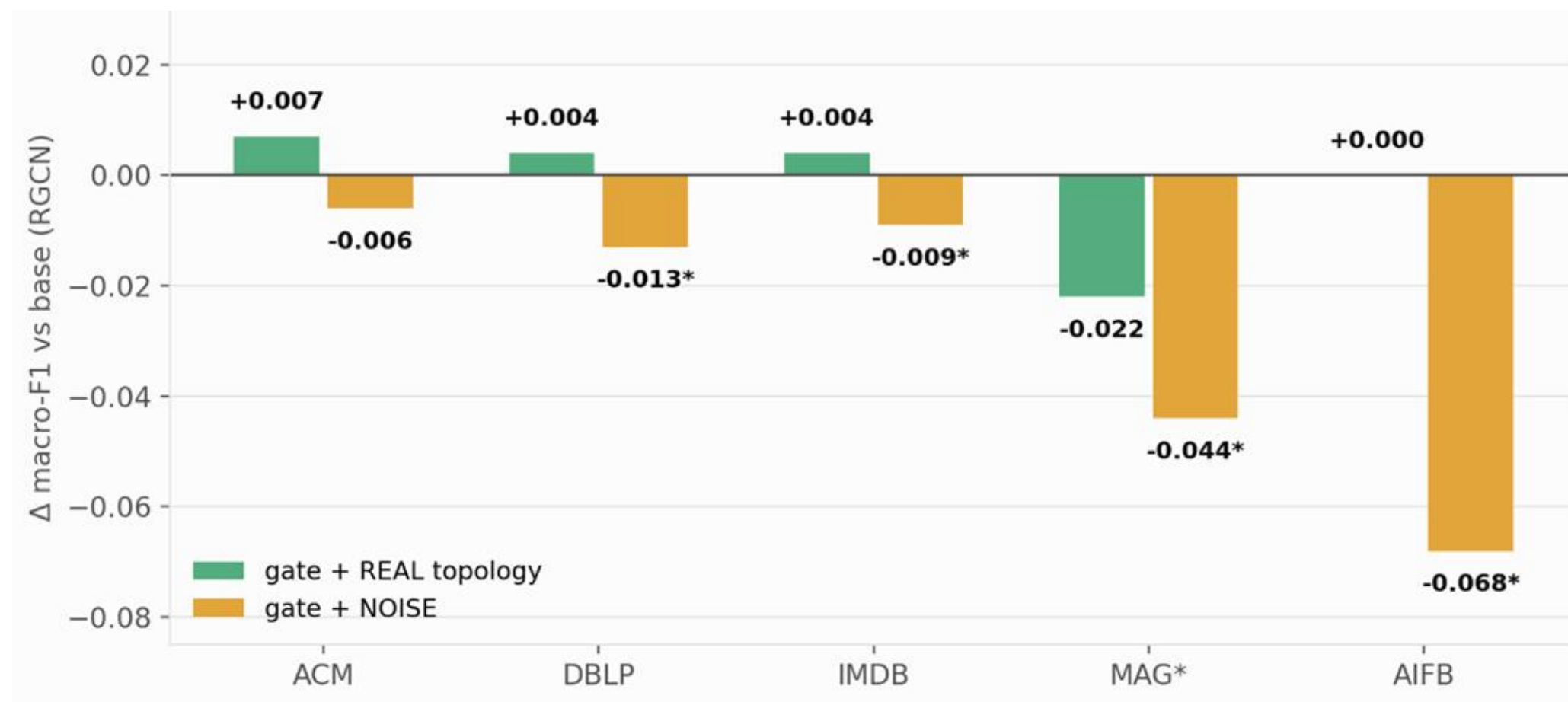


- MAG는 클래스가 많아서 클래스당 node가 ~17개 정도로 아주 적어 이러한 injection되는 feature가 학습을 방해하여 baseline보다 성능이 낮을 가능성

2. Gate가 concat의 유해함을 중화시킬 수 있다.

- gate 방식은 baseline과 동급 or 소폭 성능 향상
- gate는 구조 정보를 구조 제어에 사용했기 때문 (concat을 하게 되면 구조 상관 잡음이 발생)

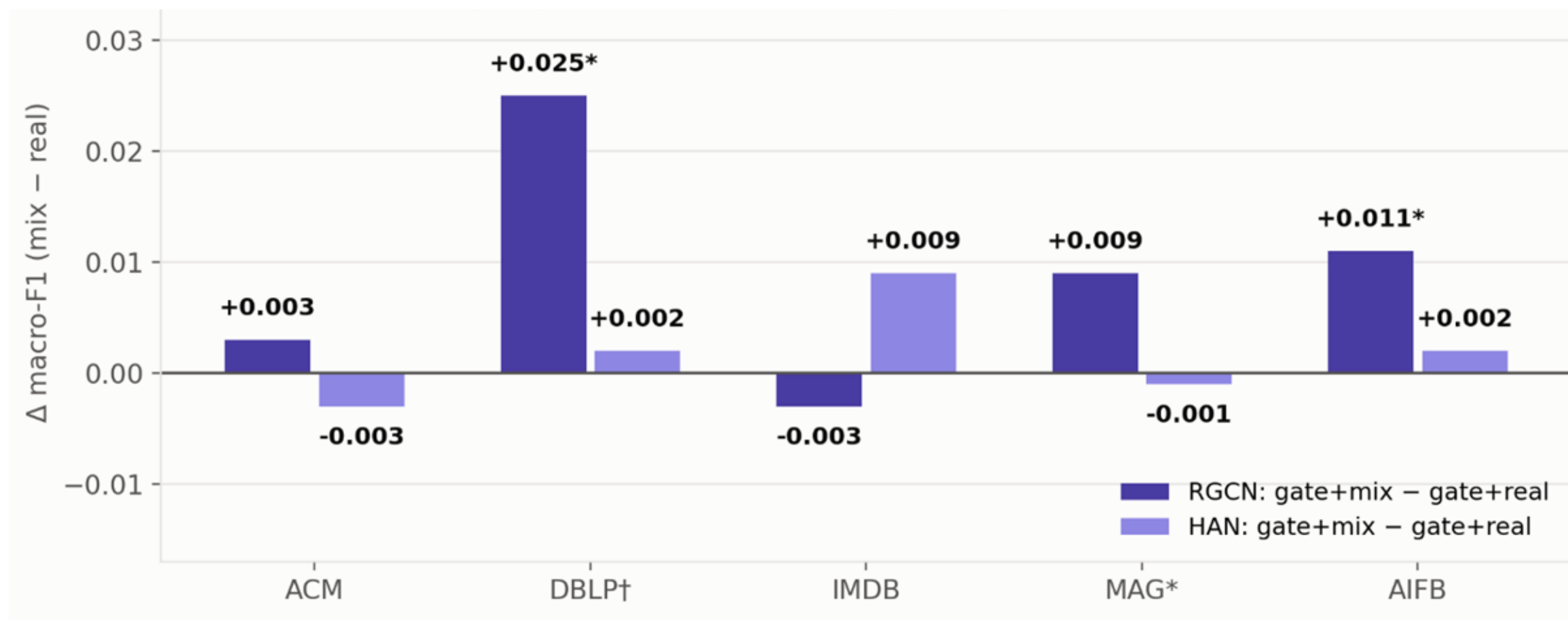
03. gate vs concat 결과



3. Gate도 정보로 위상적인 정보를 가지고 있어야지 잘 작동한다

- gate + noise는 모든 데이터에 대해 gate + real 미만
- gate는 양끝 노드의 위상을 비교하여 값을 조절하므로 입력이 진짜 위상일 때 제대로 작동하기 때문

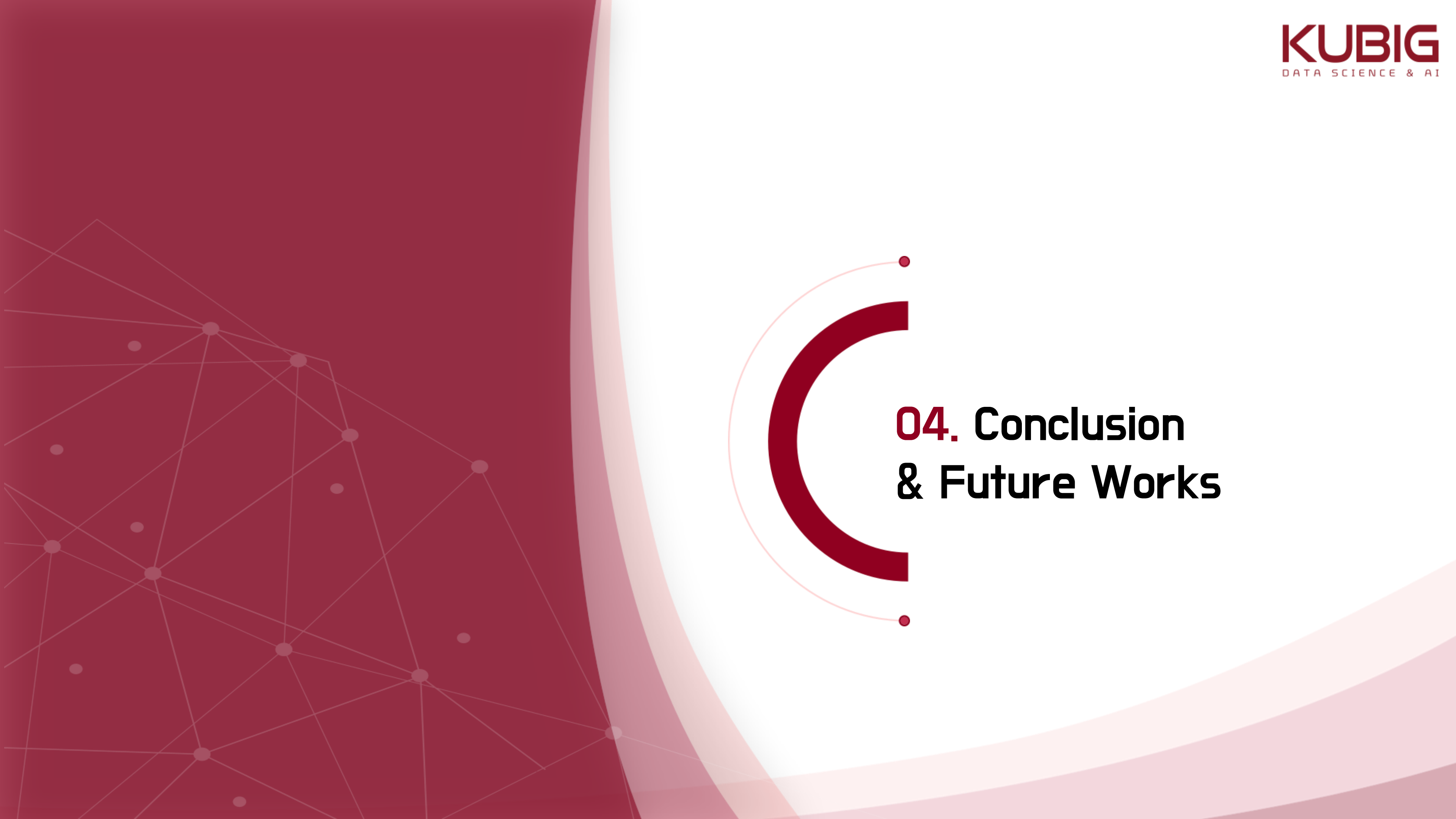
03. gate vs concat 결과



4. 하지만 node에 해당하는 정확한 위상 정보로 인한 성능 향상은 아니다.
- backbone에 상관없이 gate+mix의 성능이 gate+real과 동급이거나 오히려 높다
 - mix는 클래스 안에서만 섞기 때문에, 섞은 후에도 남는 정보는 클래스 수준의 위상 신호뿐이다. gate가 이것만으로도 real과 동급 이상으로 작동한다

03. 데이터셋별 패턴 분석

Dataset	Main Pattern	Interpretation
ACM	Concat 효과 거의 없음 Gate에서만 약한 real > mix	Feature/relation이 이미 충분함 Topology는 약한 구조 신호
DBLP	어떤 주입도 무효	클래스(연구분야)가 venue(학회)로 거의 결정 -> 메타패스의 중요성
IMDB	Concat 대부분 손해	강한 semantic feature + multilabel Topology가 noise로 작동
MAG	RGCN에서는 손해	class 많고 metric 불안정보조 증거로만 사용
AIFB	HAN/Gate에서 positive	소형 featureless graph baseline instability



04. Conclusion & Future Works

04. 결과 요약

- 1.concat 주입은 homology 정보의 올바른 injection이 아니다.
- 2.Gate가 homology 정보를 유해하지 않게 injection한다.
- 3.Gate도 노이즈가 아닌 homology 정보를 가지고 있어야지 잘 작동한다
- 4.나쁘지 않은 성능은 node에 해당하는 정확한 위상 정보 때문이 아닌 클래스 수준의 위상 신호 때문이다.

04. 결론

Q1

GTN-PDGNN을 통해 추출한 homology feature를 Heterogeneous GNN에 추가하면
Node Classification 성능이 향상되는가?

Q2

성능 향상이 있다면, 그것이 진짜 위상 신호 때문인가? 아니면 **feature dimension** 증가
효과인가?

Q3

위상 feature의 기여는 GNN backbone 구조(**HAN vs. RGCN**)에 따라 달라지는가?

04. 연구 의의 및 시사점

1.heterogeneous graph에 homology를 적용하는 파이프라인 구현

metapath로 heterogeneous graph를 homogeneous 서브그래프로 변환 → persistent homology 추출 → backbone에 연결하는 전체 흐름(GTN → PDGNN → RGCCN/HAN)을 구현. 기존에 homogeneous graph에만 쓰이던 homology를 heterogeneous graph로 확장.

2.homology 정보를 잘 반영하는 injection 방법 탐색

단순 concat에 그치지 않고 위상정보를 제대로 반영할 주입 방식을 탐색. concat vs gate 비교를 통해 구조 신호는 표현에 섞기보다 gating으로 넣는 것이 적절함을 제시.

04. 한계 및 향후 연구 과제

한계

정확한 topological 정보를 담는 방법을 찾지 못했다. node별 위상을 뽑아도 실제 유효한 건 class 수준의 거친 신호뿐이었고 ($\text{mix} \approx \text{real}$), 개별 노드의 정밀한 위상을 손실 없이 반영하는 방법이 과제로 남는다.

향후 연구 과제

link prediction으로 확장. node classification은 local feature로 충분해 global topology가 유효 신호가 되지 못했지만, link prediction은 연결 여부가 global 구조(루프·연결성)에 직접 의존하므로 homology가 기여할 여지가 크다.



Thank You